

Using Serverless Resources

(LAB-M11-01)

Overview

In this lab, you will be deploying the Lambda function, api gateway and dynamodb in VPC.

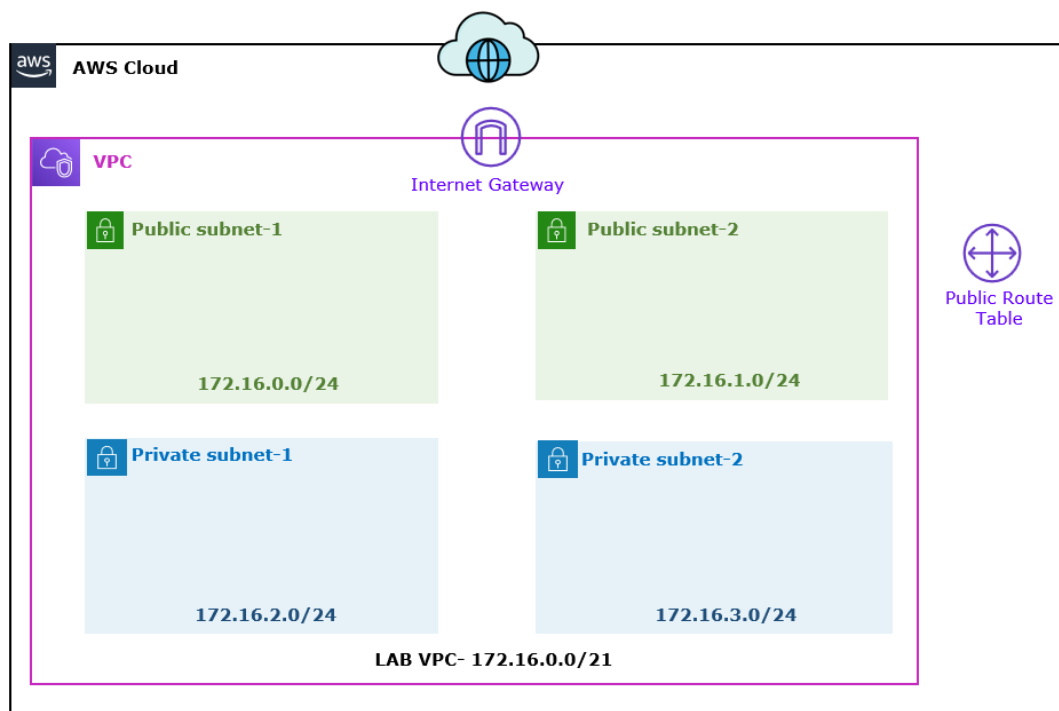
Objectives

After completing this lab, you will be able to:

- Creating DynamoDB.
- Creating Lambda function.
- Creating API Gateway.

Task 1: Create VPC and Web Server

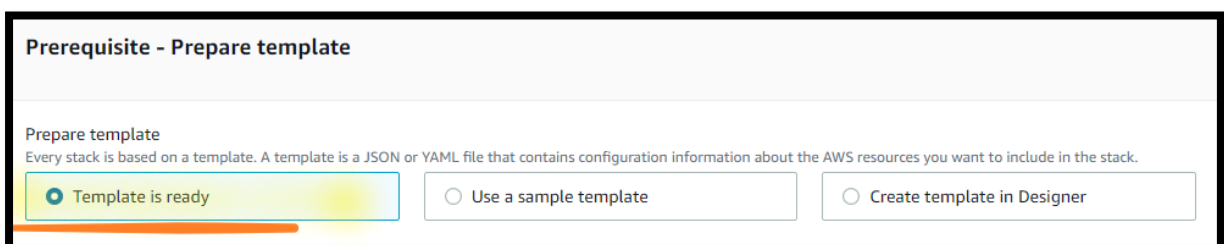
In this task of the lab, you will create the VPC and Web Server and Deploy the web app.



Step 1: Deploy a VPC using CloudFormation

AWS CloudFormation model and provision AWS resources in your cloud environment in automated way.

1. In the **AWS Management Console**, on the **Services** menu, click **CloudFormation**.
2. Dropdown and select the **US East (N. Virginia)** region.
3. Click **Create stack**:
 - a. In the **Prerequisite - Prepare template** section:
 - i. **Prepare template**: Select **Template is ready**.



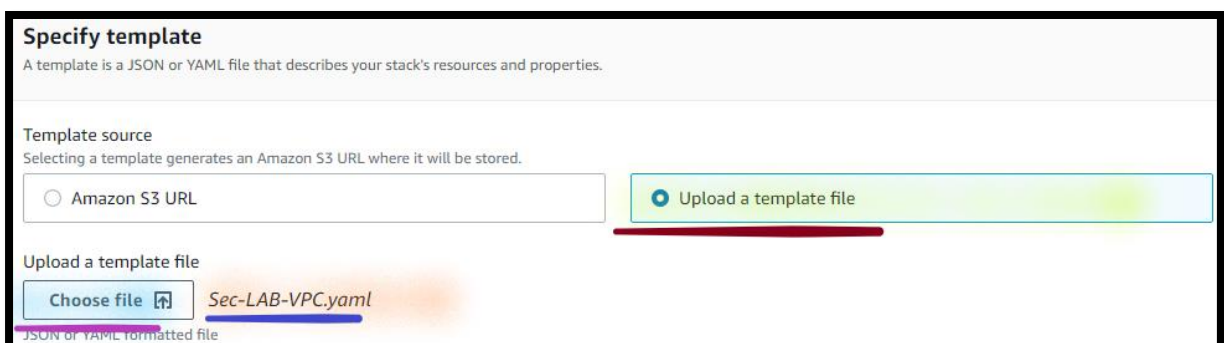
Prerequisite - Prepare template

Prepare template
Every stack is based on a template. A template is a JSON or YAML file that contains configuration information about the AWS resources you want to include in the stack.

☒ Template is ready ☐ Use a sample template ☐ Create template in Designer

- b. In the **Specify template** section:
 - i. **Template source**: Select **Upload a template file**.
 - ii. **Upload a template file**: Click on **choose file** then select the **Sec-LAB-VPC.yaml** file.

Note: **Sec-LAB-VPC.yaml** template is provided with the lab manual.



Specify template
A template is a JSON or YAML file that describes your stack's resources and properties.

Template source
Selecting a template generates an Amazon S3 URL where it will be stored.

☐ Amazon S3 URL ☒ Upload a template file

Upload a template file
Choose file **Sec-LAB-VPC.yaml**
JSON or YAML formatted file

- iii. Click **Next**.
 - c. In the **Stack name** section:
 - i. **Stack name**: Write **SecLABVPC**.

Stack name

Stack name

SecLABVPC

Stack name can include letters (A-Z and a-z), numbers (0-9), and dashes (-).

d. In the **Parameters** section:

i. **Environment name:** Write **Sec LAB VPC**.

Note: Leave other values as default.

ii. Click **Next**.

e. In the **Configure stack options** page:

i. Select **Add Tag**:

i. **Key:** Write **Name**.

ii. **Value:** Write **Sec LAB VPC**.

Note: Leave all the details as default.

ii. Click **Next**.

f. In the **Review SecLABVPC** page:

i. Click **Create stack**.

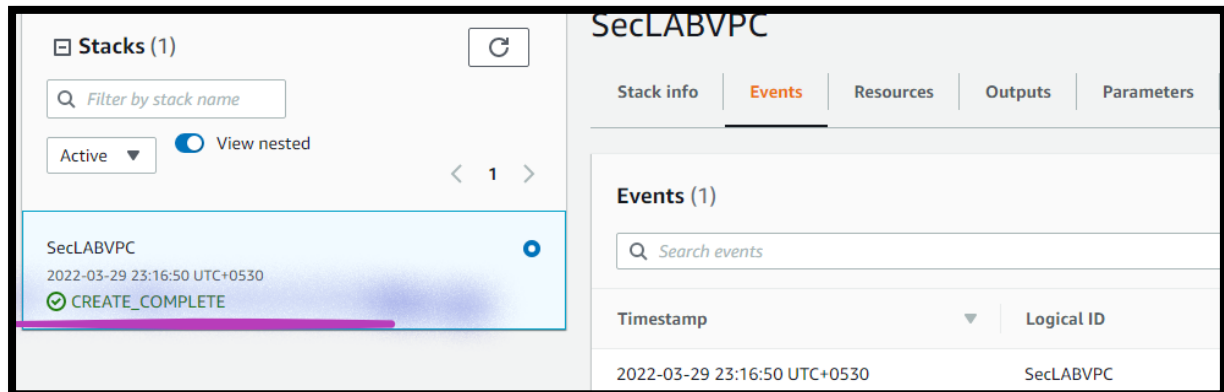
ii. The stack status (*left site under Stack*) is showing as **CREATE_IN_PROGRESS**.

The screenshot shows the AWS CloudFormation console. On the left, the 'Stacks (1)' list shows 'SecLABVPC' with a status of 'CREATE_IN_PROGRESS'. The main panel is titled 'SecLABVPC' and has tabs for 'Stack info', 'Events', 'Resources', 'Outputs', 'Parameters', 'Template', and 'Change sets'. The 'Events' tab is selected, showing a table with one event: '2022-03-22 12:58:29 UTC+0530' with status 'CREATE_IN_PROGRESS'.

Timestamp	Logical ID	Status
2022-03-22 12:58:29 UTC+0530	SecLABVPC	CREATE_IN_PROGRESS

Note: Click on **Refresh** until status is turned into **CREATE_COMPLETE**.

Note: You need to wait till you can see the **CREATE_COMPLETE** status **under Stack** on your **right site**, as shown below.



Note: **Ignore** if you can see any resource **CREATE_IN_PROGRESS** under **Events**.

4. **From the **SecLABVPC** Stack:**

- a. **Go to right**, click the **Outputs** tab.
- b. A **CloudFormation stack** can provide **output information**, such as resources details. **You will see outputs:**
 - i. **PrivateSubnets:** The **Id of the Private Subnets** that was created.
 - ii. **PublicSubnets:** The **Id of the Public Subnets** that was created.
 - iii. **VPC:** The **Id of the VPC** that was created.

SecLABVPC

Delete

Update

Stack info

Events

Resources

Outputs

Parameters

Template

Change sets

Outputs (3)

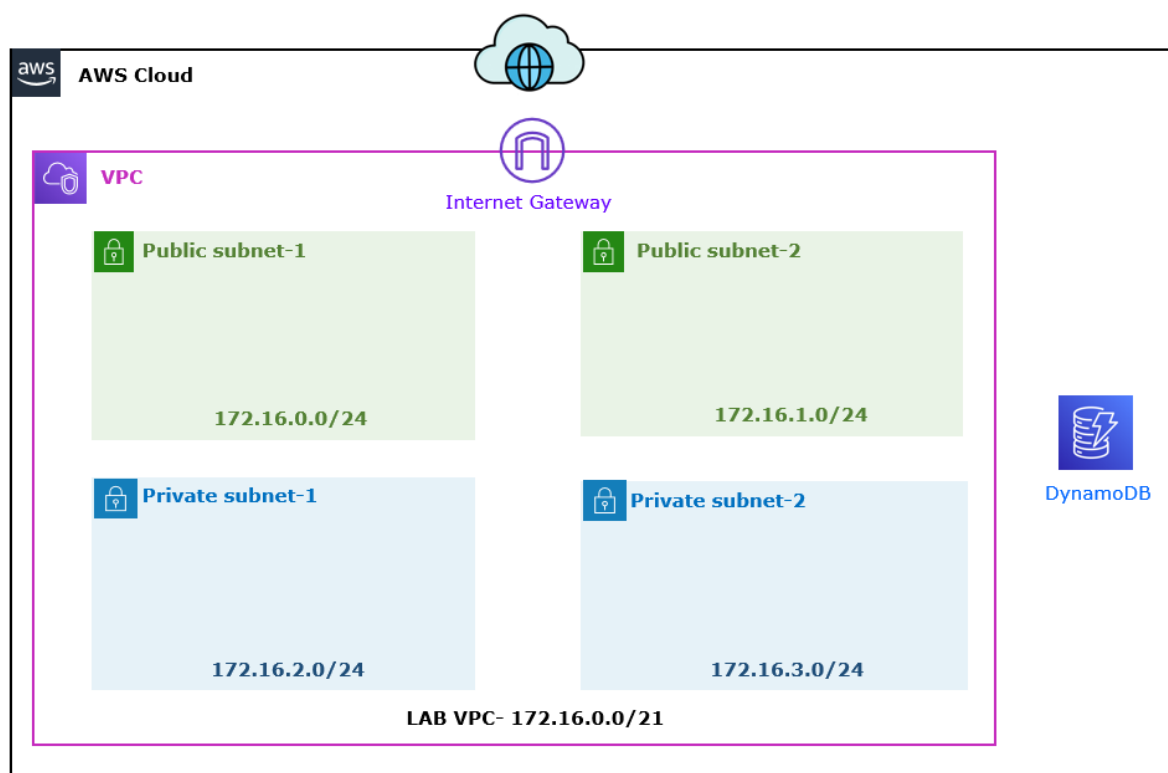
Q

Search outputs

Key	Value	Description
PrivateSubnets	subnet-0864fd776d3bad1df,subnet-0118c179dc8236559	A list of the private subnets
PublicSubnets	subnet-03189d4ab1ea884fa,subnet-0fd8b539a7987f509	A list of the public subnets
VPC	vpc-07c0650ad1b39e5a4	A reference to the created VPC

Task 2: Create Database

In this task of the lab, you will create the dynamodb table.



Step 1: Create a DynamoDB Table

5. In the **AWS Management Console**, on the **Services** menu, click **DynamoDB**.
6. Choose the **US East (N. Virginia)** region list to the right of your account information on the navigation bar.
7. Choose **Create Table** and **Configure**:
 - a. **Table name**: Write **empdata**.
 - b. **Primary key**: Write **empid**.
 - i. Set the data type to **String**.

Note: Write the table name and primary key in the **lower case** only.

- c. Select **Use default settings** under **Settings**.
- d. Select **Create**.

Note: Wait till DynamoDB table gets **created**.

Step 2: Add Items into DynamoDB Table

8. In the **AWS Management Console**, on the **Services** menu, click **DynamoDB**.
9. Select **Explore Items**.
 - a. Select **empdata**.
 - b. Select **Create Item**.

Note: New window gets opened to entered the items details from UI.

- a. **empid:** Write **001** (*in value field*).
 - i. Select **Add new attribute**.
 - ii. Select **String**.

The screenshot shows a form titled 'Attributes' with two columns: 'Attribute name' and 'Value'. The first row has 'empid - Partition key' in the 'Attribute name' column and '001' in the 'Value' column. Below this, there is a dropdown menu with the option 'Add new attribute' selected. The dropdown list shows 'String' and 'Number' as options. The 'String' option is highlighted with a blue bar, and the 'Number' option is highlighted with a red bar.

- b. **Attribute name:** Write **empfirstname**.
 - i. **Value:** Write **Ajay**.
 - 1) Select **Add new attribute**.
 - 2) Select **String**.
- c. **Attribute name:** Write **emplastname**.
 - i. **Value:** Write **Kaushik**.
 - 1) Select **Add new attribute**.
 - 2) Select **String**.
- d. **Attribute name:** Write **empage**.
 - i. **Value:** Write **32**.

Note: Write the **empfirstname**, **emplastname** and **empage** in the **lower case** only.

Attribute name	Value
empid - Partition key	001
empfirstname	Ajay
emplastname	Kaushik
empage	32

e. Select **Create Item**.

Note: You can view the newly created item details under Items.

Items returned (1)					Actions ▼	Create item
Find items					< 1 >	⚙️
☐	empid ▼	empage ▼	empfirst... ▼	emplastname ▼		
☐	001	32	Ajay	Kaushik		

Task 2: Create First Lambda Function

In this task of the lab, you will create the lambda function.

Step 1: Create IAM Role

10. In the **AWS Management Console**, on the **Services** menu, click **IAM**.

11. Select **Roles**, click on **Create role**.

12. Select **Lambda**, under a **use case**.

a. Select **Next: Permissions**.

i. Search and select **AmazonDynamoDBFullAccess**.

ii. Search and select **AWSLambdaBasicExecutionRole**.

iii. Search and select **AmazonEC2FullAccess**.

Note: Lambda uses your function's permissions to create and manage network interfaces. To connect to a VPC, your function's execution role must have the EC2 NetworkInterface access.

- b. Select **Next: Tags**.
- c. Select **Next: Review**.

Note: Here you will see **DynamoDB Policy** under policies.

- i. **Role name:** Write **Lambda-DynamoDB-Role**.
- d. Click **Create role**.

Note: You get the message, the role **Lambda-DynamoDB-Role** been created.

Step 2: Create Security Group for Lambda

In this task of the lab, you will create the Security group for the lambda function.

13. In the **AWS Management Console**, on the **Services** menu, click **EC2**.

14. **Go to the left**, select **Security Groups**.

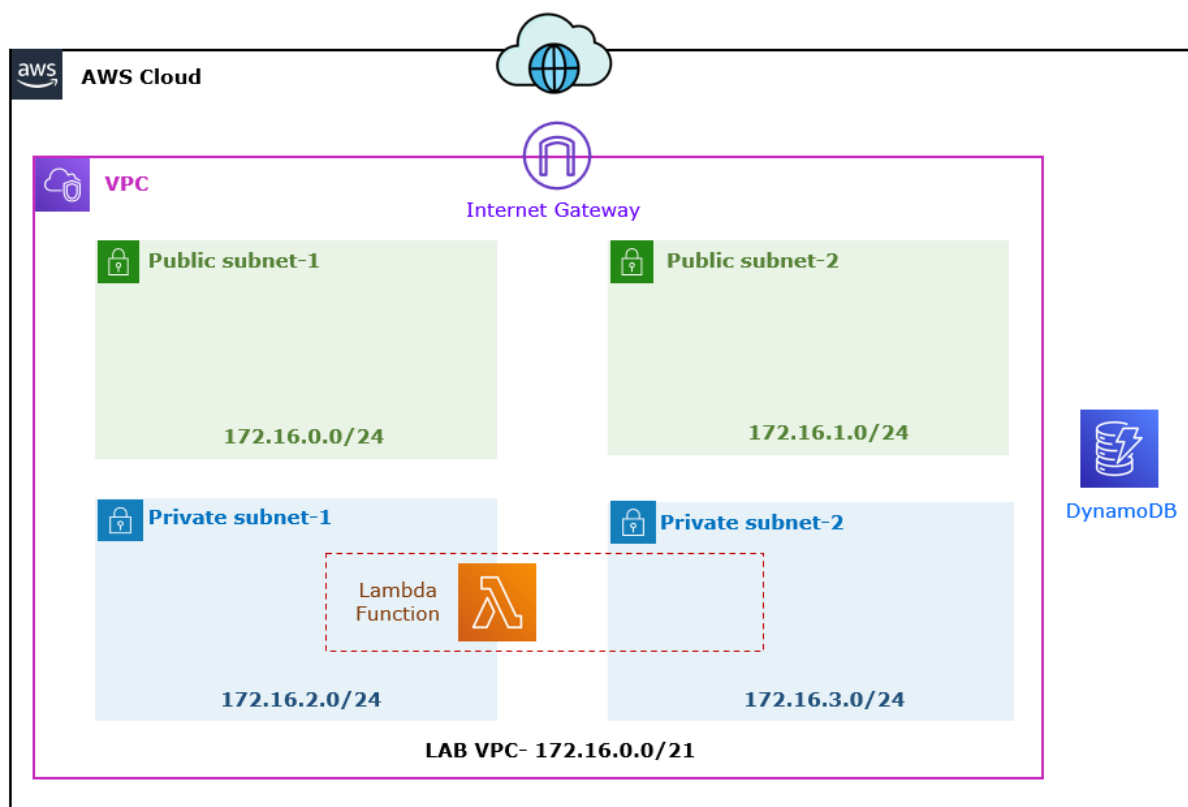
15. Select **Create security group**.

- a. In the **Basic details** section:
 - i. **Security group name:** Write **Lambda-SG**.
 - ii. **Description:** Write **Security Group for Lambda**.
 - iii. **VPC:** Click on **Cross**.
 - a) Select **Sec LAB VPC**.
- b. In the **Inbound rules** section:
 - i. Click **Add Rule**.
 - a) **Type:** Dropdown and select **HTTP**.
 - b) **Source:** Dropdown and select **Custom**.

- 1) In the **Search Box**, type **172.16.2.0/22**.
- c) **Description: Write Allow HTTP Traffic from Private Subnets.**
- c. In the **Tags - optional** section:
 - i. Click **Add new tag**:
 - a) **Key**: Write **Name**.
 - b) **Value**: Write **Lambda-SG**.
 - d. Click **Create security group**.

Step 3: Create Lambda Function to Read the Items

In this task of the lab, you will create the lambda function in the VPC.



16. In the **AWS Management Console**, on the **Services** menu, click **Lambda**.

17. Click **Create a function**.

18. Select **Author from scratch** and configure:

- a. **Name:** Write **ReadItems**.
- b. **Runtime:** Dropdown and Select **Node.js [Latest version]**.



Function name
Enter a name that describes the purpose of your function.

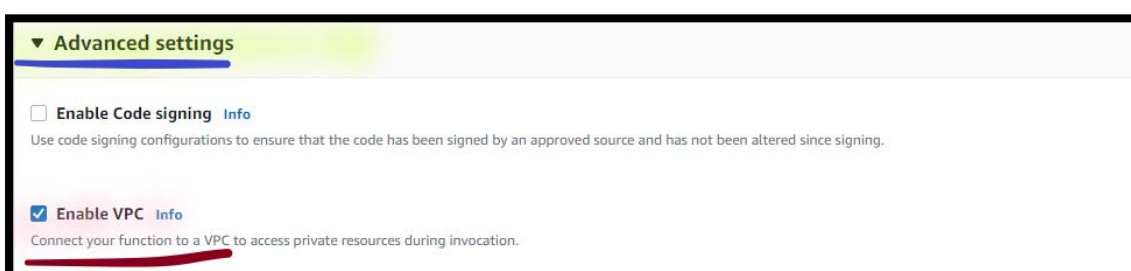
ReadItems

Use only letters, numbers, hyphens, or underscores with no spaces.

Runtime [Info](#)
Choose the language to use to write your function.

Node.js 10.x

- c. **Expand** *Change default execution role.*
 - i. **Role:** Select **Use an existing role**.
 - ii. **Existing role:** Dropdown and Select **Lambda-DynamoDB-Role**.
- d. **Expand** *Advanced settings.*
 - i. Select **Enable VPC**.



▼ **Advanced settings**

☐ **Enable Code signing** [Info](#)
Use code signing configurations to ensure that the code has been signed by an approved source and has not been altered since signing.

☒ **Enable VPC** [Info](#)
Connect your function to a VPC to access private resources during invocation.

- ii. **VPC:** Dropdown and select **Sec LAB VPC**.
- iii. **Subnets:** Dropdown and select **Private Subnet-1** and **Private Subnet-2**.

iv. **Security groups:** Dropdown and select **Lambda-SG**.

e. Select **Create function**.

Note: ReadItems function gets open the configuration section.

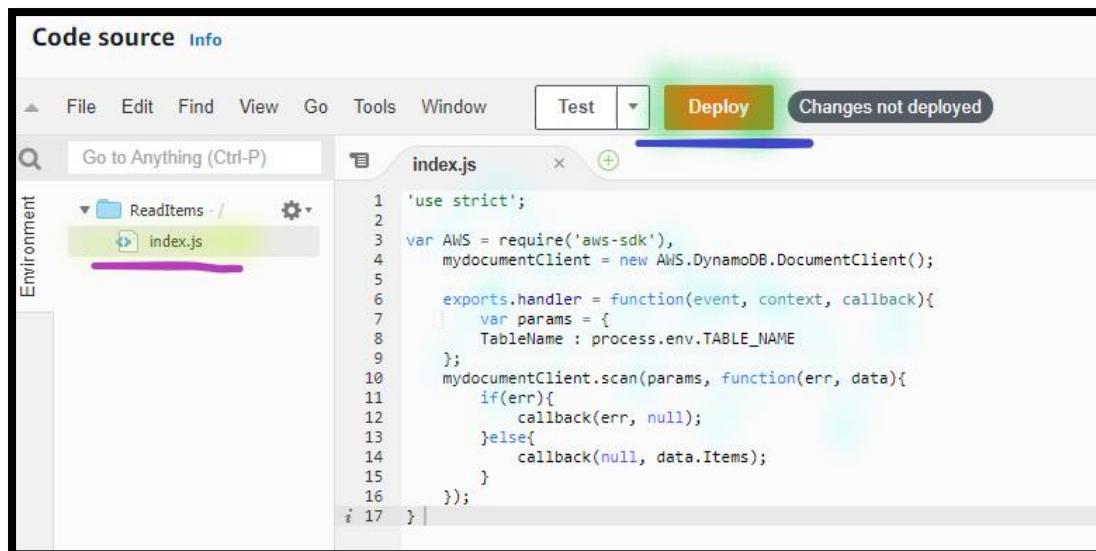
Note: Wait (~5 mnts.) till ReadItems function gets created.

19. **Select** the **Code** section:

- a. Click on **index.js**.
 - i. **Replace** the **existing code** and **copy** the code into the **editor** from **Read-Items-Lambda-Function-Code.txt** file.

Note: **Code** for **Read-Items-Lambda-Function-Code.txt** is available with the Lab manual.

- b. Select **Deploy**.



20. **Select** the **Configuration** section:

- Select **Environment variables**.
- Select **Edit**.



c. Select **Add environment variables**.

- Key:** Write **TABLE_NAME**.
- Value:** Write **empdata** (DynamoDB table name).

Environment variables

You can define environment variables as key-value pairs that are accessible from your function. You can store configuration settings without the need to change function code. [Learn more](#)

Key: Value:

d. Select **Save**.

Step 2: Validate Your Implementation

21. **Select** the **Test** section:

- Template:** Dropdown and Select **hello-world**.
- Name:** Write **TestReadItems**.
- In the **Event**, **Remove** the existing events and **copy** the below event:

```
{  
  "empid": "*"   
}
```

Test event

Invoke your function with a test event. Choose a template that matches the service that triggers your function.

☒ New event ☐ Saved event

Template:

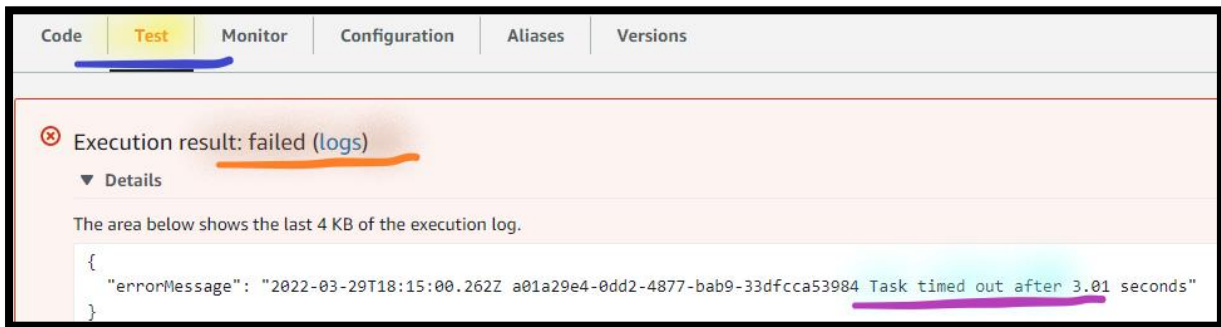
Name:

```
1 {  
2   "empid" : "*"   
3 }  
4
```

d. Select **Test**.

Note: Once you Test the function and code executed successfully you can see the execution result as **Failed** with **Timed Out**.

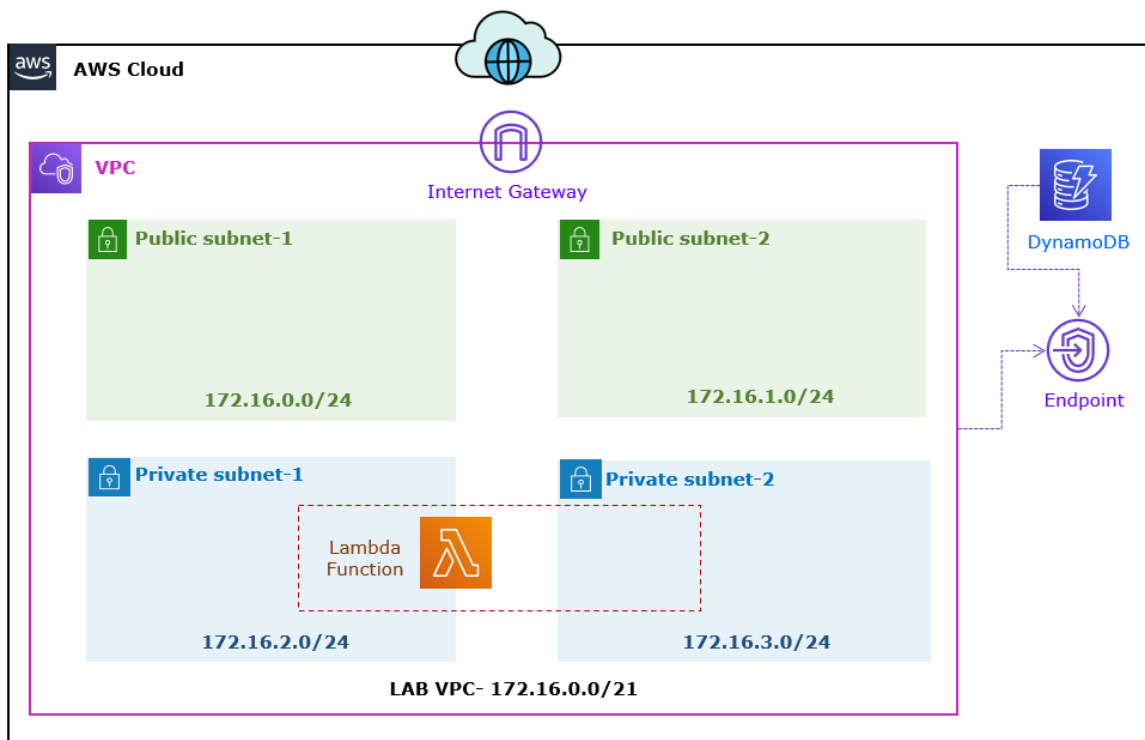
- e. **Expand** the **Details** section of the **execution result** section.



Note: Reason for **Timed Out**, because **no connectivity** between Lambda deployed in Private Subnet with DynamoDB.

Step 3: Create VPC Endpoint for DynamoDB

In this task of the lab, you will create the vpc endpoint for dynamodb to enable the secure communication between lambda and dynamodb.



22. In the **AWS Management Console**, on the **Services** menu, click **VPC**.

23. **Go to the left**, select **Endpoints**.

24. Select **Create Endpoint**.

a. In the **Endpoint settings** section:

i. **Name tag**: Write **DynamoDB Endpoint**.

ii. **Service category**: Select **AWS services**.

The screenshot shows the 'Create Endpoint' page in the AWS Management Console, specifically the 'Endpoint settings' section. It includes a 'Name tag - optional' field with the value 'DynamoDB Endpoint'. Below this is the 'Service category' section with four radio button options: 'AWS services' (selected), 'PrivateLink Ready partner services', 'AWS Marketplace services', and 'Other endpoint services'.

b. In the **Services** section:

i. In the **search box** type **com.amazonaws.us-east-1.dynamodb** and Press **enter**.

Note: You can see the **com.amazonaws.us-east-1.dynamodb** after search.

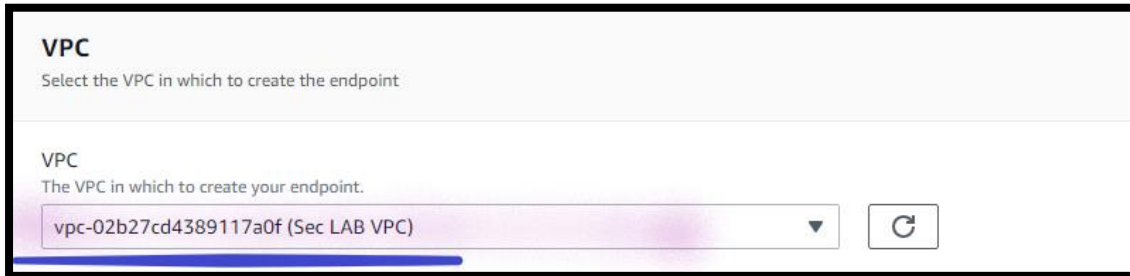
ii. Select **com.amazonaws.us-east-1.dynamodb**.

The screenshot shows the 'Services' section of the AWS Management Console. A search filter 'com.amazonaws.us-east-1.dynamodb' is applied. The results table shows one entry: 'com.amazonaws.us-east-1.dynamodb' with owner 'amazon' and type 'Gateway'.

Service Name	Owner	Type
com.amazonaws.us-east-1.dynamodb	amazon	Gateway

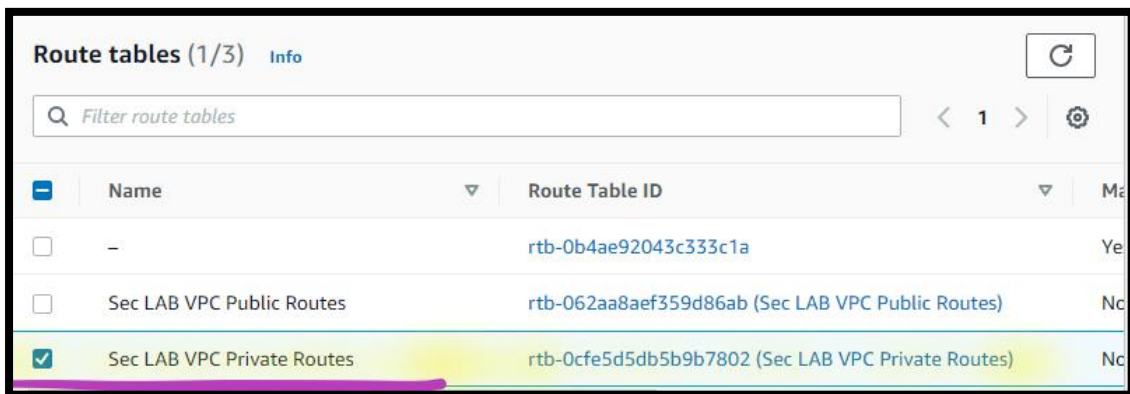
c. In the **VPC** section:

i. **VPC**: Dropdown and select **Sec LAB VPC**.



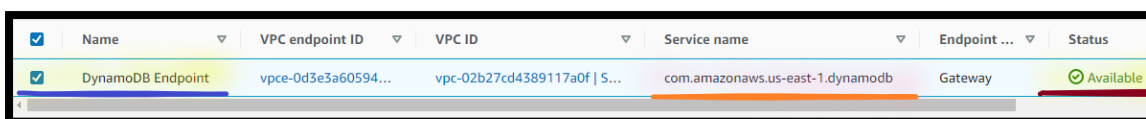
d. In the **Route tables** section:

i. Select the **Sec LAB VPC Private Routes**.



e. Select **Create endpoint**.

Note: Wait till, Service endpoint **Status** shown as **Available**.



Step 3: Validate Lambda Implementation

25. In the **AWS Management Console**, on the **Services** menu, click

Lambda.

26. Select **Functions**.

27. Open **ReadItems** functions.

a. **Select** the **Test** section:

- i. **Template:** Dropdown and Select **hello-world**.
- ii. **Name:** Write **TestReadItems**.
- iii. In the **Event**, **Remove** the existing events and **copy** the below event:

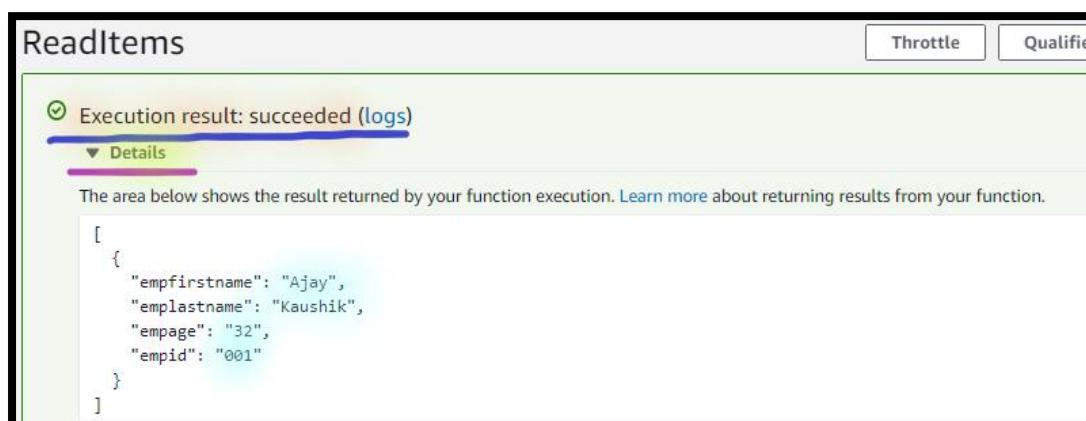
```
{  
  "empid": "*"   
}
```

- b. Select **Test**.

Note: Once you Test the function and code executed successfully you can see the execution result as **succeeded**.

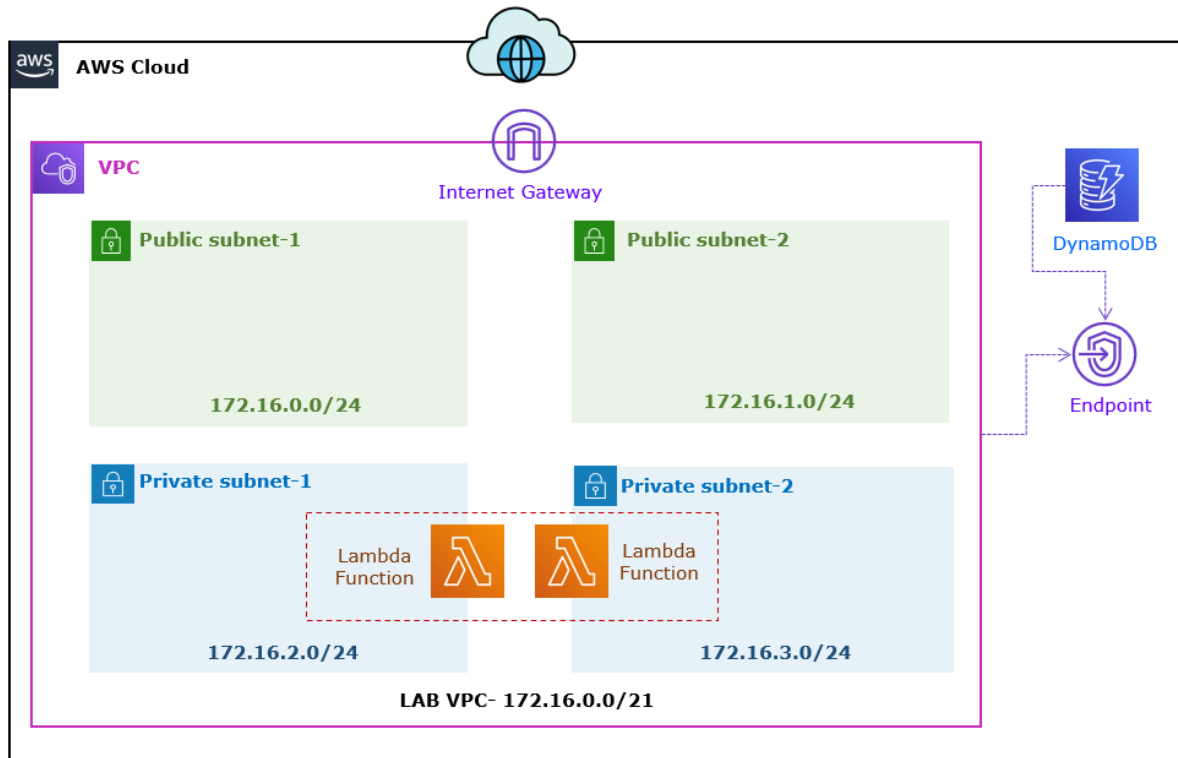
- c. **Expand** the **Details** section of the **execution result** section.

Note: You can view the **Items**, which you have added in the DynamoDB in the Previous Step.



Task 4: Create Second Lambda Function

In this task of the lab, you will create the second lambda function in the VPC.



Step 1: Create Lambda Function to Write the Items

28. In the **AWS Management Console**, on the **Services** menu, click **Lambda**.

29. Click **Create a function**.

30. Select **Author from scratch** and configure:

- a. **Name:** Write **WriteItems**.
- b. **Runtime:** Dropdown and Select **Node.js [Latest version]**.
- c. **Expand** *Change default execution role.*
 - i. **Role:** Select **Use an existing role**.
 - ii. **Existing role:** Dropdown and Select **Lambda-DynamoDB-Role**.
- d. **Expand** *Advanced settings.*
 - i. Select **Enable VPC**.
 - ii. **VPC:** Dropdown and select **Sec LAB VPC**.

- iii. **Subnets:** Dropdown and select **Private Subnet-1** and **Private Subnet-2**.
- iv. **Security groups:** Dropdown and select **Lambda-SG**.
- e. Select **Create function**.

Note: WriteItems function gets open the configuration section.

Note: Wait (~5 mnts.) till WriteItems **function** gets **created**.

31. **Select** the **Configuration** section:

- a. Select **Environment variables**.
- b. Select **Edit**.
- c. Select **Add environment variables**.
 - i. **Key:** Write **TABLE_NAME**.
 - ii. **Value:** Write **empdata** (DynamoDB table name).
- d. Select **Save**.

32. Select the **Code** section:

- a. Click on **index.js**.
 - i. **Replace** the **existing code** and **copy** the code into the **editor** from **Write-Items-Lambda-Function-Code.txt** file.

Note: **Code** for **Write-Items-Lambda-Function-Code.txt** is available with the Lab manual.

- b. Select **Deploy**.

Step 2: Validate Your Implementation

33. **Select** the **Test** section:

- a. **Template:** Dropdown and Select **hello-world**.
- b. **Name:** Write **TestWriteItems**.

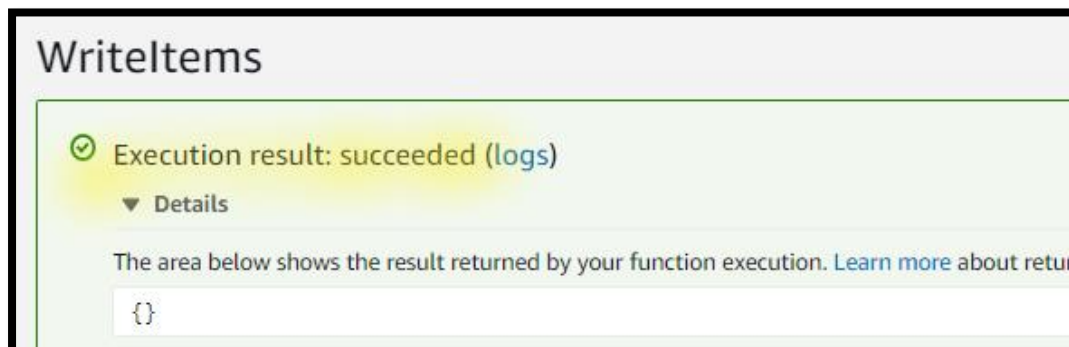
- c. In the **Event**, **Remove** the existing events and **copy** the below event:

Note: You can add the new items now.

```
{
  "empid": "002",
  "empfirstname": "Sana",
  "emplastname": "Yusuf",
  "empage": "21"
}
```

- d. Select **Test**.

Note: Once you Test the function and code executed successfully you can see the Execution result as **succeeded**.



Step 3: View the DynamoDB Data

34. In the **AWS Management Console**, on the **Services** menu, click **DynamoDB**.
35. Select **Explore Items**.
- Select **empdata** DynamoDB table.
 - Select **Run**.

Note: You can view the **Added Items**, which you have added in the DynamoDB via the **Lambda function**.

Run

Reset

✔️ Completed

Read capacity units consumed: 0.5

Items returned (2)

Actions ▼

Create item

Find items

< 1 >

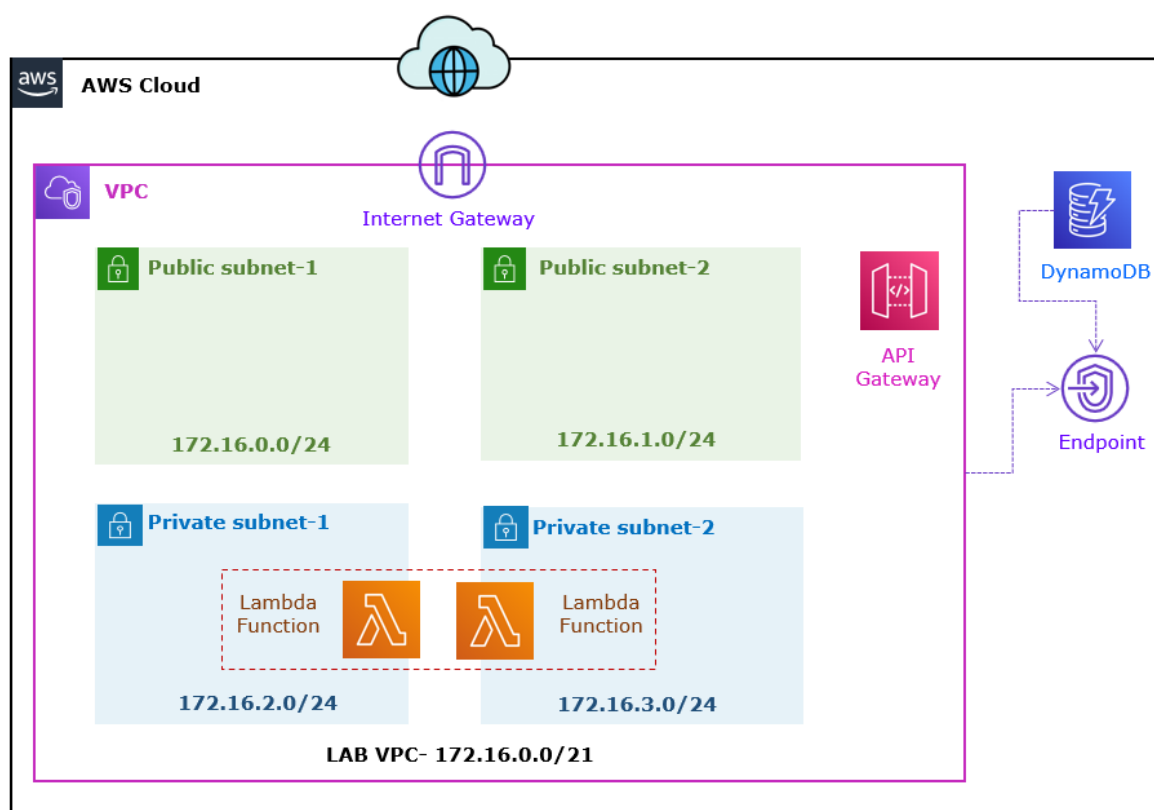
⚙️

✖️

☐	empid ▼	empage ▼	empfirst... ▼	emplastname ▼
☐	001	32	Ajay	Kaushik
☐	002	21	Sana	Yusuf

Task 5: Create a RESTful API

In this task of the lab, you will create the API gateway to secure your backend resources.



Step 1: Create Security Group for API Gateway

36. In the **AWS Management Console**, on the **Services** menu, click **EC2**.

37. **Go to the left**, select **Security Groups**.

38. Select **Create security group**.

a. In the **Basic details** section:

iv. **Security group name**: Write **APIGW-SG**.

v. **Description**: Write **Security Group for API Gateway**.

vi. **VPC**: Click on **Cross**.

a) Select **Sec LAB VPC**.

b. In the **Inbound rules** section:

i. Click **Add Rule**.

d) **Type**: Dropdown and select **HTTPS**.

e) **Source**: Dropdown and select **Custom**.

1) In the **Search Box**, type **172.16.2.0/23**.

f) **Description**: Write **Allow HTTP Traffic from Public Subnets**.

c. In the **Tags - optional** section:

i. Click **Add new tag**:

a) **Key**: Write **Name**.

b) **Value**: Write **Lambda-SG**.

d. Click **Create security group**.

Step 2: Create VPC Endpoint for API Gateway

39. In the **AWS Management Console**, on the **Services** menu, click **VPC**.

40. **Go to the left**, select **Endpoints**.

41. Select **Create Endpoint**.

- a. In the **Endpoint settings** section:
 - i. **Name tag**: Write **APIGateway Endpoint**.
 - ii. **Service category**: Select **AWS services**.
- b. In the **Services** section:
 - i. In the **search box** type **com.amazonaws.us-east-1.execute-api** and Press **enter**.

Note: You can see the **com.amazonaws.us-east-1.execute-api** after search.

- ii. Select **com.amazonaws.us-east-1.execute-api**.
- c. In the **VPC** section:
 - i. **VPC**: Dropdown and select **Sec LAB VPC**.

Note: You will now specify which *subnets* the should **privately** access **API Gateway**.

- d. In the **Subnets** section:
 - a) Select the **First Availability Zone**:
 - a) Dropdown and select **Private subnet-1**.
 - ii. Select the **Second Availability Zone**:
 - a) Dropdown and select **Private subnet-2**.
- e. In the **Security group** section:
 - a) Select the **APIGW-SG** security group.
- f. Select **Create endpoint**.

Note: **Wait** till, Service endpoint **Status** shown as **Available**.

- g. **Copy** the **VPC endpoint ID** in the **Notepad**.

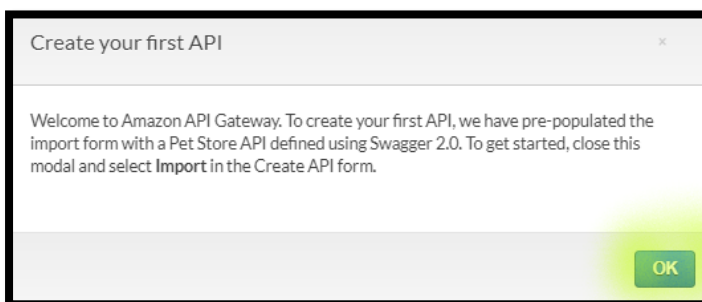
✓	Name	VPC endpoint ID	VPC ID	Service name	Endpoint ...	Status
✓	APIGateway Endpoint	vpce-09b1d69cff81a3feb	vpc-02b27cd4389117...	com.amazonaws.us-east-1.execute-api	Interface	Available

Step 3: Create REST API

42. In the **AWS Management Console**, on the **Services** menu, click **API Gateway**.
43. Choose the **US East (N. Virginia)** region list to the right of your account information on the navigation bar.
44. Select **Rest API Private**.
 - a. Select **Build**.



- b. Select **OK**, when **Create your first API** window prompt.



- c. Select **New API** under **Create new API**.
 - d. In the **Settings**, provide the following:
 - i. **API name**: Write **empdata-API**.
 - ii. **Endpoint type**: Dropdown and select **Private**.
 - iii. **VPC Endpoint IDs**: Write **VPC endpoint ID** (which you have copied in the previous step)

API name* empdata-API

Description

Endpoint Type Private

Private APIs are only accessible through VPC endpoints for API Gateway. Create a VPC endpoint and add Resource Policy for the API to allow access. [Learn more.](#)

VPC Endpoint IDs

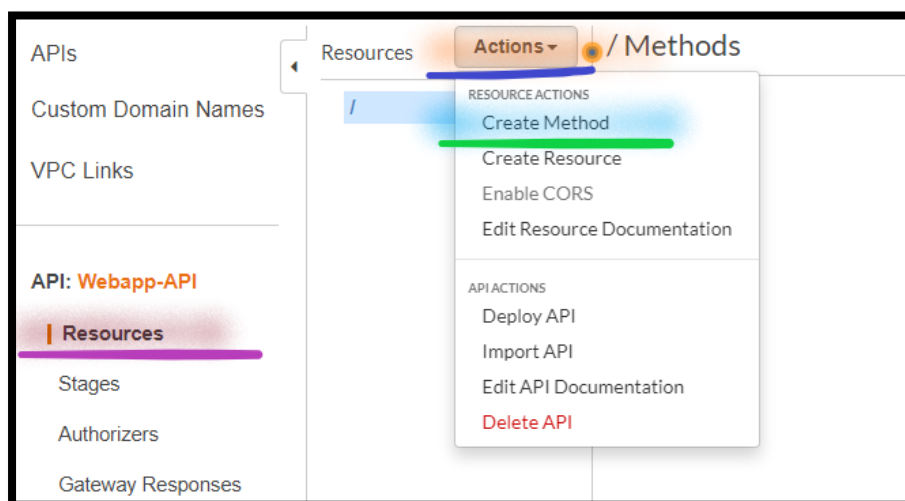
- vpce-09b1d69cff81a3feb [X]

- e. Select **Create API**.

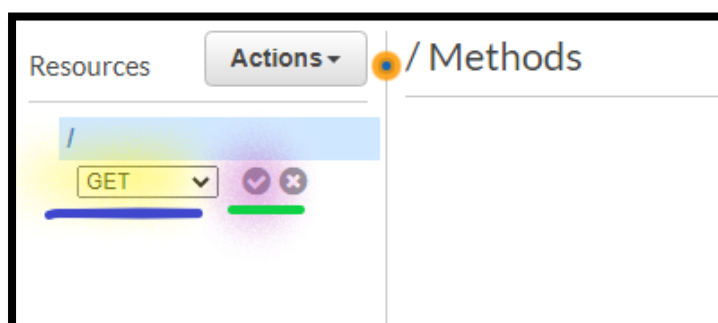
Step 4: Create Method to Read the Items

45. Go to left, choose **Resources**.

- a. From the **Actions** dropdown select **Create Method**.



- b. Select **Get** from the new dropdown that appears, then click the **checkmark**.



- c. **Integration type:** Select **Lambda Function**.
- d. **Lambda region:** Dropdown and Select the **us-east-1** region.
- e. **Lambda function:** Write **ReadItems**, the lambda function you created in the previous lab, that read the data from DynamoDB table.

Note: Leave other details as default.

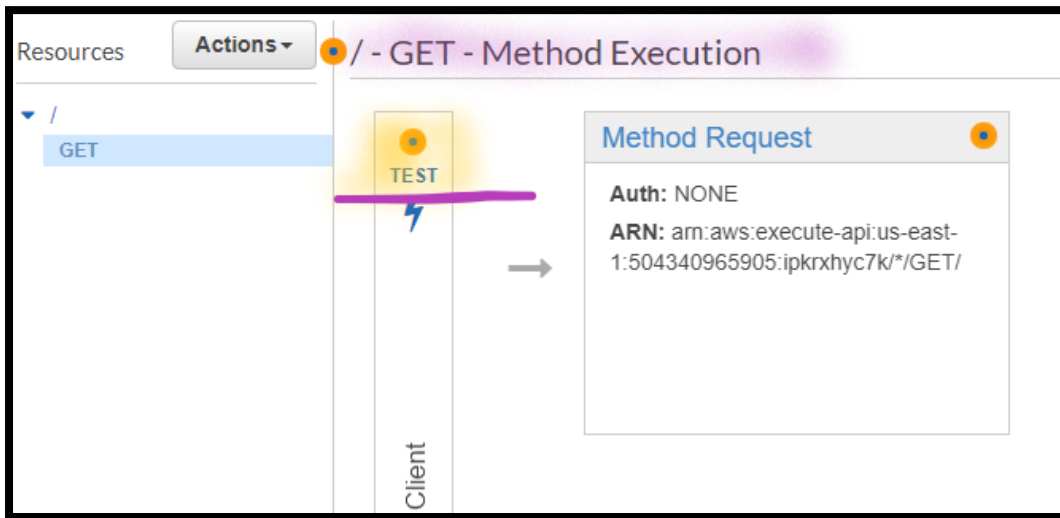
The screenshot shows the 'Choose the integration point for your new method' dialog in the AWS API Gateway console. The 'Integration type' is set to 'Lambda Function'. The 'Use Lambda Proxy integration' checkbox is unchecked. The 'Lambda Region' dropdown is set to 'us-east-1'. The 'Lambda Function' dropdown is set to 'ReadItems'. The 'Use Default Timeout' checkbox is checked.

- f. Select **Save**.
- g. When **prompted** to give Amazon API Gateway permission to invoke your function, choose **OK**.

The screenshot shows the 'Add Permission to Lambda Function' dialog. It contains the text: 'You are about to give API Gateway permission to invoke your Lambda function: arn:aws:lambda:us-east-1:504340965905:function:RequestRide'. At the bottom right, there are 'Cancel' and 'OK' buttons.

Step 5: Test the API Gateway to Read the Items

46. Click on **Test** under **GET - Method Execution**.



a. Click the **Test**.

Note: If request executed **successfully**, you can see the Request status as **200**.

Note: In the Response body, you can see the **Items**, which you have added in the DynamoDB in the previous step.

Step 6: Create Method to Write the Items

47. **Go to left**, choose **Resources**.

48. From the **Actions** dropdown select **Create Method**.

49. Select **Post** from the new dropdown that appears, then click the **checkmark**.

- a. **Integration type:** Select **Lambda Function**.
- b. **Lambda region:** Dropdown and Select the **us-east-1** region.
- c. **Lambda function:** Write **WriteItems**, the lambda function you created in the previous lab, that write the data into DynamoDB table.

Note: Leave other details as default.

Choose the integration point for your new method.

Integration type ☒ Lambda Function ⓘ

☐ HTTP ⓘ

☐ Mock ⓘ

☐ AWS Service ⓘ

☐ VPC Link ⓘ

Use Lambda Proxy integration ☐ ⓘ

Lambda Region us-east-1 ▼

Lambda Function WriteItems

Use Default Timeout ☒ ⓘ

- d. Select **Save**.
- e. When **prompted** to give Amazon API Gateway permission to invoke your function, choose **OK**.

Add Permission to Lambda Function

You are about to give API Gateway permission to invoke your Lambda function:
arn:aws:lambda:us-east-1:504340965905:function:RequestRide

Cancel OK

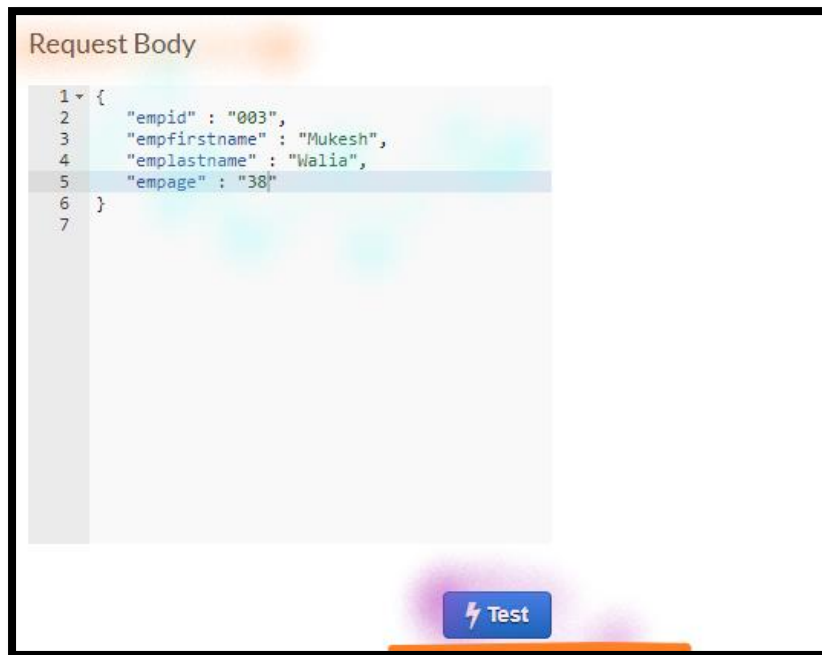
Step 7: Test the API Gateway to Write the Items

50. Click on **Test** under **Post - Method Execution**.
 - a. Click the **Test**.
 - b. In the **Request Body**, **Copy** the below event:

Note: You can now add the new items via API Gateway.

```
{
  "empid": "003",
  "empfirstname": "Mukesh",
  "emplastname": "Walia",
  "empage" : "38"
}
```

- c. Click the **Test**.



Note: If request executed **successfully**, you can see the Request status as **200**.

Note: You can view the **Items** in DynamoDB, which you have added in the DynamoDB via the **API gateway**.

Step 8: View the DynamoDB Data

51. In the **AWS Management Console**, on the **Services** menu, click **DynamoDB**.

52. Select **Explore Items**.

- a. Select **empdata** DynamoDB table.
- b. Select **Run**.

Note: You can view the **Added Items**, which you have added in the DynamoDB via the **API gateway**.

empid	empage	empfirst...	emplastname
001	32	Ajay	Kaushik
003	38	Mukesh	Walia
002	21	Sana	Yusuf

Step 9: Configure the Resource Policy

53. In the **AWS Management Console**, on the **Services** menu, click **API Gateway**.

54. Open **empdata-API** API.

55. Go to left, choose **Resource Policy**.

a. **Update** the **API Gateway Policy**.

Note: **API Gateway Policy.txt** is available with the Lab manual.

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": "*",
7       "Action": "execute-api:Invoke",
8       "Resource": "arn:aws:execute-api:us-east-1:504340965905:pq66ecxpjd/*"
9     },
10    {
11      "Effect": "Deny",
12      "Principal": "*",
13      "Action": "execute-api:Invoke",
14      "Resource": "arn:aws:execute-api:us-east-1:504340965905:pq66ecxpjd/*",
15      "Condition": {
16        "StringNotEquals": {
17          "aws:SourceVpc": "vpc-02b27cd4389117a0f"
18        }
19      }
20    }
21  ]
22 }
```

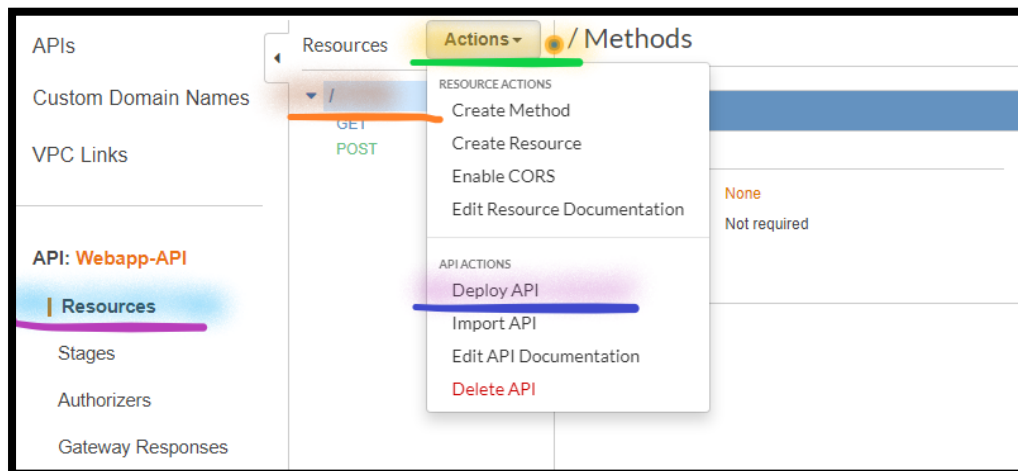
Note: Replace the **SEC-LAB-VPC_ID** with **Sec LAB VPC ID**.

b. Select **Save**.

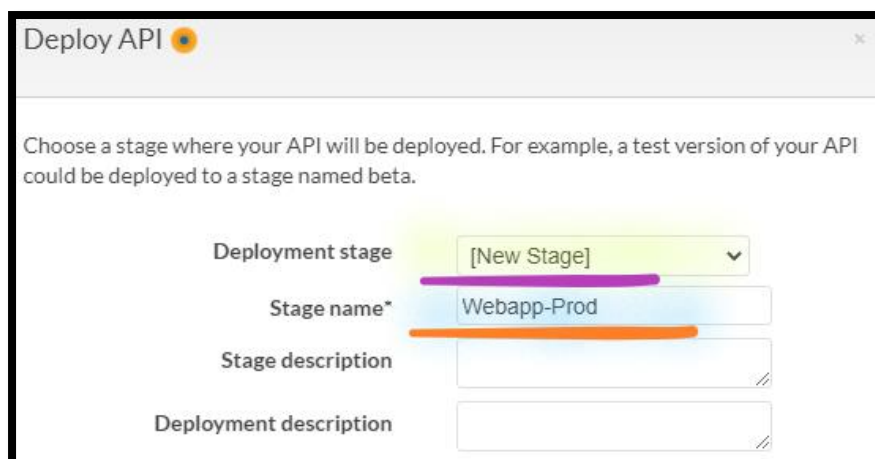
Step 10: Deploy API

56. Go to left, choose **Resources**.

- Select **/ resource**.
- From the **Actions** dropdown select **Deploy API**.

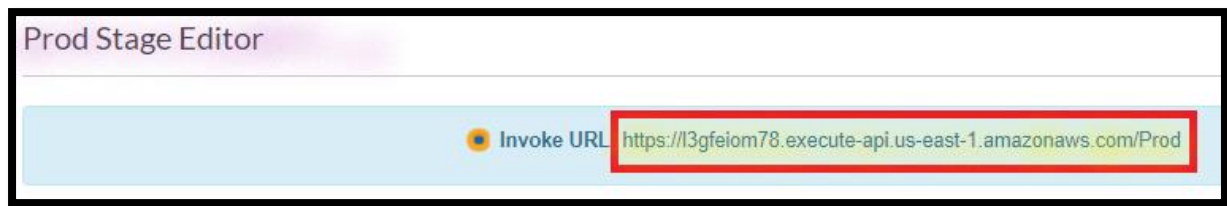


- Deployment stage:** Dropdown and Select **[New Stage]**.
- Stage name:** Write **ReadWrite-API**.



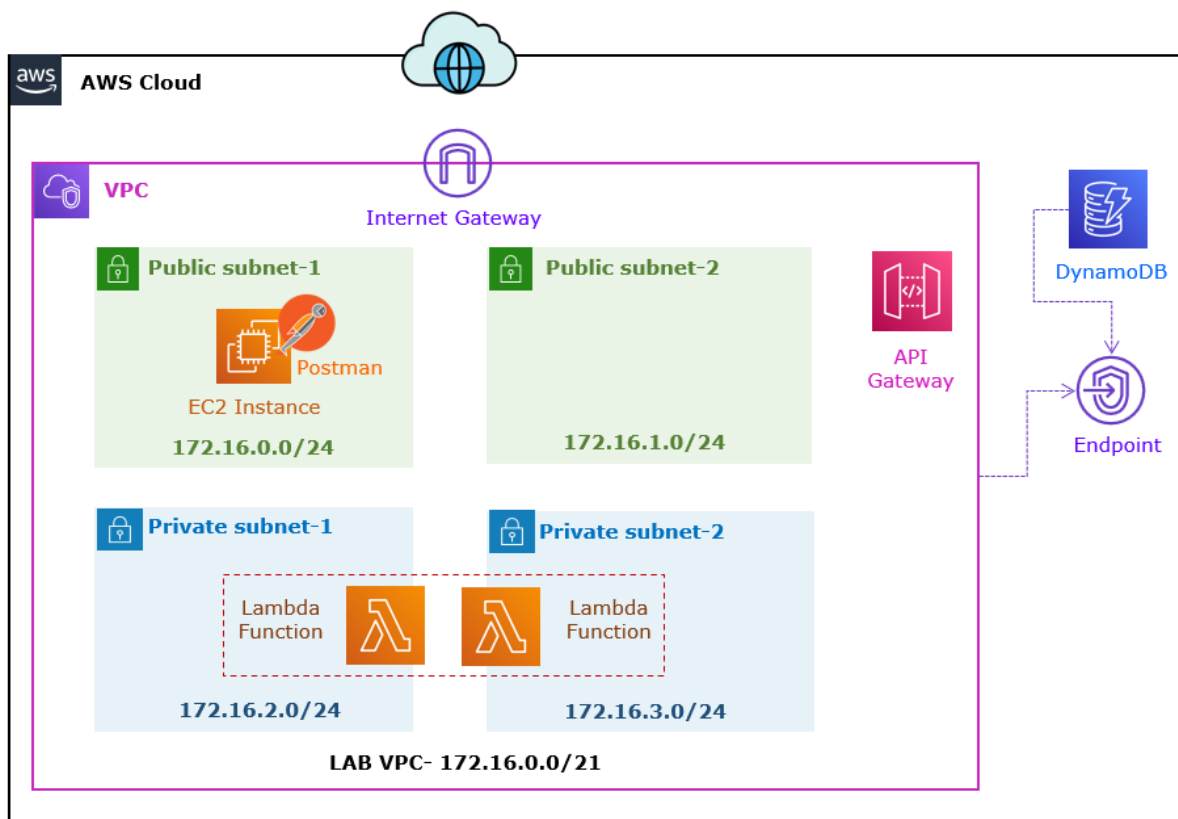
- Select **Deploy**.

57. Copy the **Invoke URL** in the **Notepad**.



Task 5: Validate the Solution using Postman

In this task of the lab, you will test the API gateway.



Step 1: Create Windows Server

58. In the **AWS Management Console**, on the **Services** menu, click **EC2**.

59. Click **Launch Instance**.

- a. In the **Choose an Amazon Machine Image (AMI)** page:
 - i. Select the **Microsoft Windows Server 2019 Base**.
 - ii. Click **Select**.

- b. In the **Choose an Instance Type** page:
 - i. Select the **t2.micro**.
 - ii. Click **Next: Configure Instance Details**.
- c. In the **Configure Instance Details** page:
 - i. **Network:** Dropdown and select **Sec LAB VPC**.
 - ii. **Subnet:** Dropdown and select **Public Subnet-1**.

Note: Leave the other settings as default.

- iii. Select **Next: Add Storage**.
- d. In the **Add Storage** page:

Note: Leave all the settings as default.

- i. Select **Next: Add Tags**.
- e. In the **Add Tags** page:
 - i. Click on **Add Tag**:
 - a) **Key:** Write **Name**.
 - b) **Value:** Write **Windows Server**.
 - ii. Select **Next: Configure Security Group**.
- f. In the **Configure Security Group** page:
 - i. **Assign a security group:** Select **Create a new security group**.
 - a) **Security group name:** Write **WindowsWebServer-SG**.
 - b) **Description:** Write **Windows Web Server Security Group**.

Info: **RDP Port [3389]**, already added and allowed from anywhere [0.0.0.0/0].

- ii. Select **Review and Launch**.
- g. In the **Review Instance Launch** page:
 - i. Select **Launch**.
 - a) In the **Select an existing key pair or create a new key pair** section:
 - 1) **First selection box:** Dropdown and select **Choose an existing key pair**.
 - 2) **Key pair name:** Dropdown and select **My-SEC-LAB-KP**.
 - 3) **Enable** the **Checkmark** against the ***I acknowledge that I have access***

Note: Leave the other settings as default.

- ii. Click on **Launch Instances**.
- h. Select the **View Instances**.

Step 2: Check the Web Server Status

- 60. **From** the **EC2** console:
- 61. Click **Instance**.
 - a. Select **Web Server**.
 - i. **Wait** for the **Instance State** to change to **Running state**.
 - ii. **Wait** for the **Status check** to change to **2/2 checks passed**.

Note: **Wait** (~5-10 mnts.) for the server deployment.

- b. Select **Web Server**.
 - i. **Go below** in the console and click on **Details**.
 - ii. **Expand** the **Instance summary**.
 - a) **Copy** the **Public IP address** in the **Notepad**.

Step 3: Generate Windows Password

62. To generate windows password, select **Windows Server** virtual machine.

- a. Select **Actions**.
- b. Select **Security**.
- c. Select **Get Windows Password**.
 - i. **Browse**: Navigate and Select **My-Sec-LAB-KP.pem** key pair.
 - ii. Click on **Decrypt Password**.

Note: Windows will pop-up with **user name** and **password**.

Note: Copy the **user name** and **password** in **Notepad**.

- d. Select **Close**.

Step 4: Remote Desktop from Windows Desktop/ Laptop

63. **From** the **local Desktop/ Laptop**, right click on **Start** & **Run**.

64. In the **Open**, write **mstsc**, press **Ok**.

- a. **Type** the **Public IP Address** of the **Windows Server** instance.
- b. Click **Connect**.
- c. **Type** the **Username** and **Password** of the **Windows Server** instance which you copied in the previous step and click **Ok**.
- d. Click on **Yes** to confirm this connection, if prompted with the security message.

Step 5: Install the Dot Net SDK

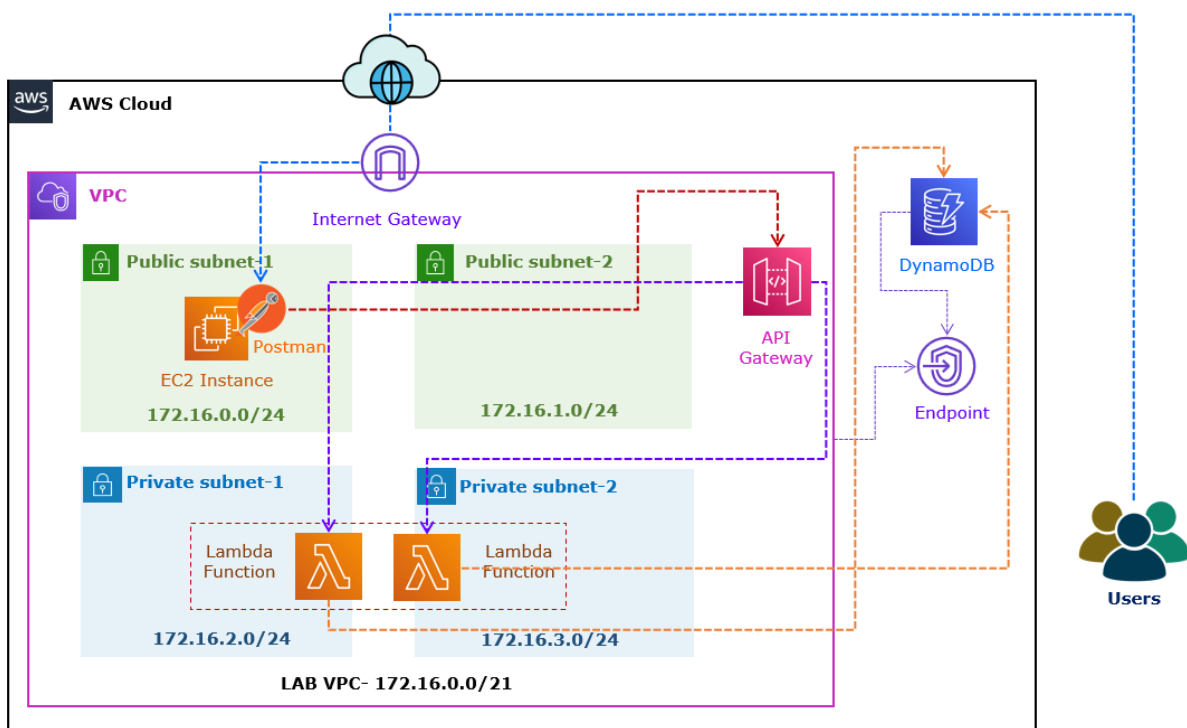
65. From the **Windows Server**, right click on **Start** & **Run**.
- Go to **Start menu**, open **Server manager**.
 - Select **Local Server**.
 - Click in **On** showing against **IE Enhanced Security Configuration**.
 - Select **Off** in Administrator and Select **Ok**.
 - Refresh** your screen & now you can see **Off** showing against **IE Enhanced Security Configuration**.
 - Download** and **Install** the **Postman**.

Note: Use the below URL to download the **Postman for Windows**.

<https://www.postman.com/downloads/>

Note: Wait, till **Postman** install **successfully**.

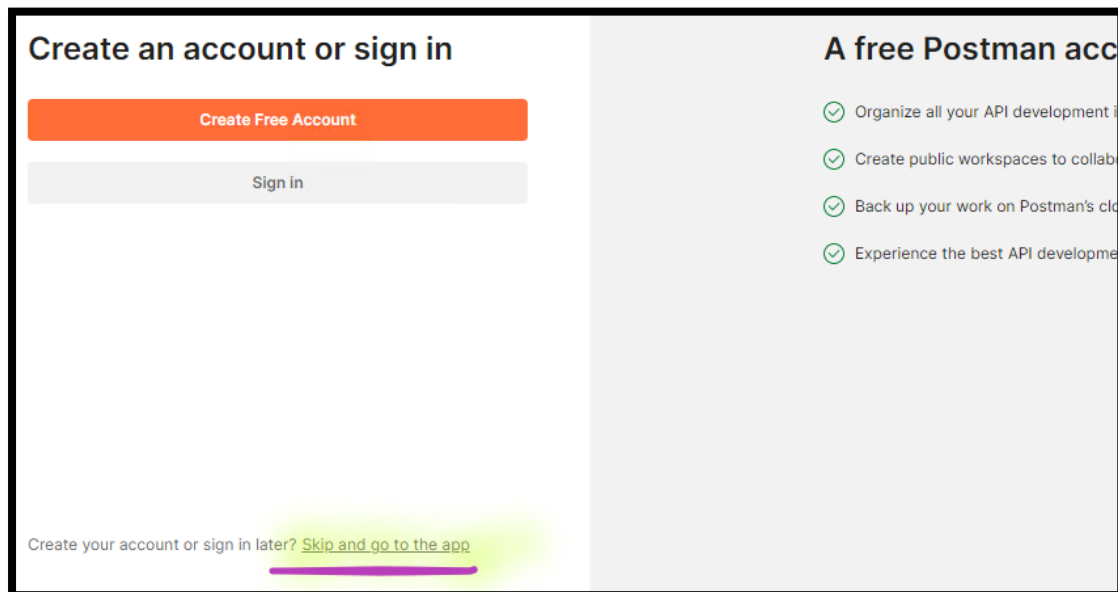
Step 6: Test the Azure **ReadItems** function



66. **From** the **Windows Server**:

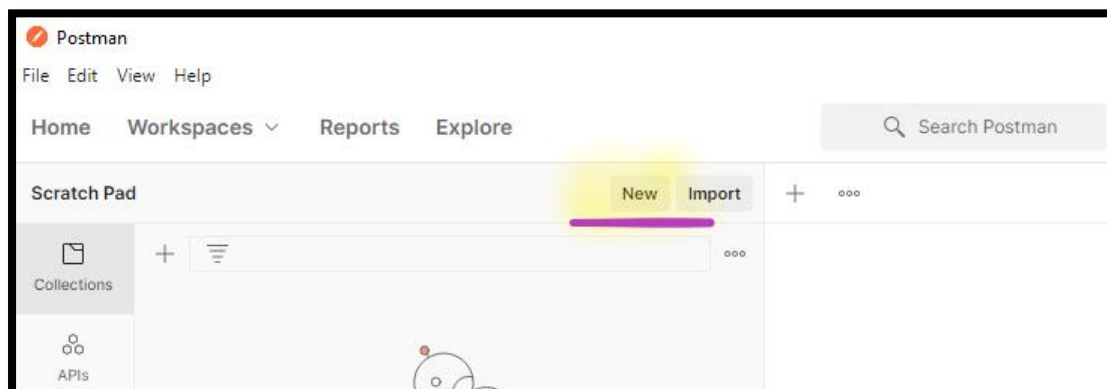
67. Open the **Postman**.

Note: You can select the **Skip and go to app** to use Postman without Login into the Postman.

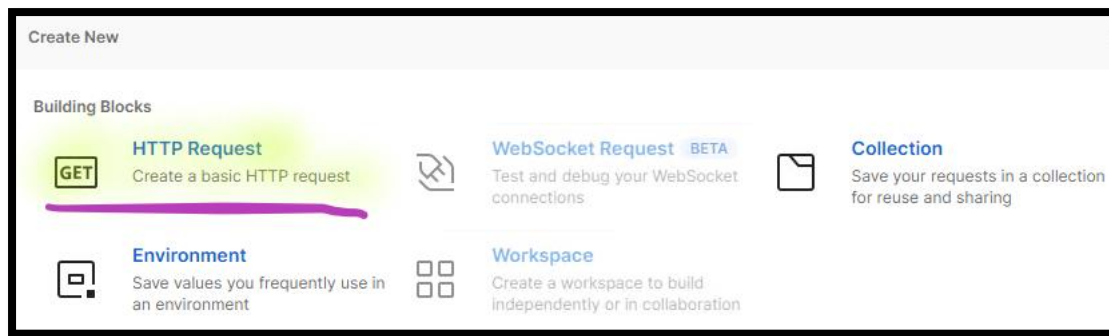


68. **From** the **Postman**:

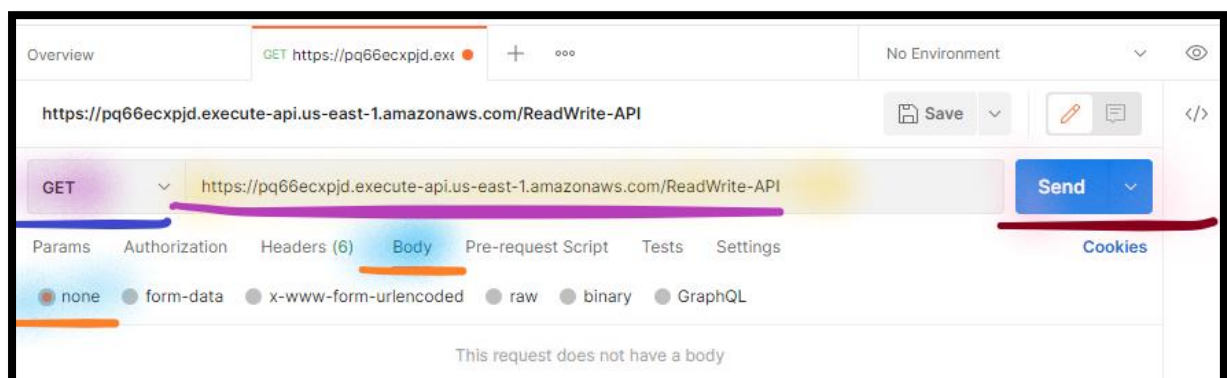
a. Select **New**.



b. From **Popup window**, select **HTTP request**.



- a. From the **Request** section:
 - i. Dropdown and select **Get**.
 - ii. **Enter Request URL:** **Copy** the **InvokeURL**.
 - iii. Select **Body**.
 - iv. Select **None**.
 - v. Select **Send**.



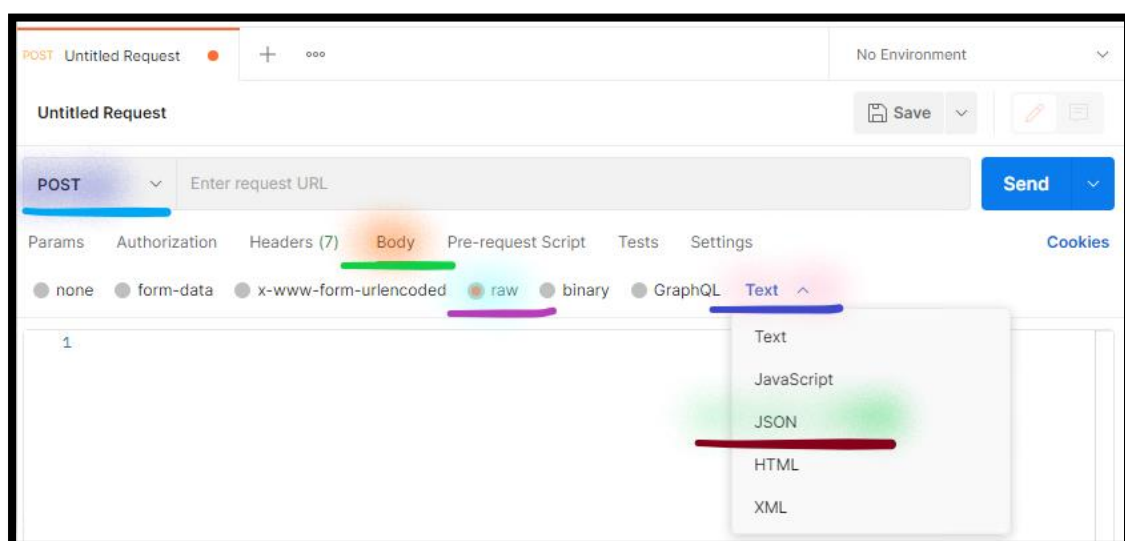
Note: If data fetch successfully, you can see the response.



Step 7: Test the Azure **WriteItems** function

69. From the **Postman**:

- a. From the **Request** section:
 - i. Dropdown and select **Post**.
 - ii. Select **Body**.
 - iii. Select **Raw**.
 - iv. Select **Text**:
 - 1) Dropdown and select **JSON**.



70. **From** the **Postman Request** section:

- a. **Enter Request URL:** **Copy** the **InvokeURL**.
- b. **Body:** **Add** the *below JSON*:

```
{  
  "empid": "004",  
  "empfirstname": "Rajeev",  
  "emplastname": "kumar",  
  "empage": "45"  
}
```

- c. Select **Send**.

Step 8: View the DynamoDB Data

71. In the **AWS Management Console**, on the **Services** menu, click **DynamoDB**.

72. Select **Explore Items**.

- a. Select **empdata** DynamoDB table.
- b. Select **Run**.

Note: You can view the **Added Items**, which you have added in the DynamoDB via the **Postman**.

Task 5: Delete the Environment

Step 1: Delete the DynamoDB Table

73. In the **AWS Management Console**, on the **Services** menu, click **DynamoDB**.

74. Click the **Tables**.

- a. Select the **emdpdata**.
- b. Select **Delete table**.

Step 2: Delete Lambda Function

75. In the **AWS Management Console**, on the **Services** menu, click **Lambda**.

76. Click the **Functions**.

- a. Select the **ReadItems**.
- b. Select **Actions**.
- c. Select **Delete**.

77. Click the **Functions**.

- a. Select the **WriteItems**.
- b. Select **Actions**.
- c. Select **Delete**.

Step 3: Delete the API Gateway

78. In the **AWS Management Console**, on the **Services** menu, click **API Gateway**.

79. Select the **empdata-API**.

- a. Click on the **Actions**.
- b. Select the **Delete**.
 - a) To confirm deletion, type **delete**.
 - b) Select the **Delete**.

Step 4: Terminate EC2 Instances

80. In the **AWS Management Console**, on the **Services** menu, click **EC2**.

81. Click **Instances**.

82. Select **Windows Server**.

- i. Click on **Instance state**.
- ii. Select **Terminate instance**.
- iii. Select **Terminate**.

Step 5: Delete Private Endpoint

7. In the **AWS Management Console**, on the **Services** menu, click **VPC**.
8. Open the **Endpoints**.
 - a. Select **DynamoDB Endpoint** and **APIGateway Endpoint**.
 - i. Select **Actions**.
 - ii. Select **Delete VPC Endpoint**.
 - a) To confirm deletion, type **delete**.
 - iii. Select **Delete**.

Step 6: Delete Security Groups

9. In the **AWS Management Console**, on the **Services** menu, click **VPC**.
10. Select the **Security groups**.
 - a. Select the **Lambda-SG** and **APIGW-SG**.
 - i. Select **Actions**.
 - ii. Select **Delete security groups**.
 - iii. To confirm deletion, type **delete**.
 - iv. Choose **Delete**.

Step 7: Delete the Stack

11. In the AWS Management Console, on the **Services** menu, click **CloudFormation**.
 - a. Select **Stack**.
 - i. Select **SecLABVPC**.
 - a) Select **Delete**.
 - b) Select **Delete stack**.